

Day 5

8/4 화

새로고침해도 안 사라지게 만드는 날

01

localStorage 로 데이터를 저장하고 다시 불러올 수 있다

02

상태를 바꾸고 화면을 다시 그리는 구조로 코드를 짤 수 있다

03

중단점을 찍어 코드가 멈춘 자리에서 값을 직접 확인할 수 있다

오늘의 흐름

새 문법은 적음. 지금까지 쓴 것을 구조로 묶는 날

오전, 이론 09:00 - 12:50

■ 09:00	새로고침하면 왜 사라지나
■ 10:00	상태 하나, 화면 하나
■ 11:00	콘솔 말고 중단점
■ 12:00	AI 코드 고치기

오후, 실습 14:00 - 17:50

■ 14:00	미션 브리핑, 함께 작성
■ 15:00	개인 개발, 중간 풀이
■ 16:00	개인 개발, 중간 풀이
■ 17:00	추가 문제, 마무리

01 새로고침하면 왜 사라지나

나를 내내 같은 질문을 드렸습니다

매일 마지막에 한 번씩 물었던 것

- 01 Day 1 할 일 목록. 새로그침하니 다 없어졌습니다
- 02 Day 2 대시보드. 정렬을 바꿔봐도 새로 열면 처음으로
- 03 Day 3 룰렛. 기록이 안 남습니다
- 04 Day 4 날씨. 방금 받아온 기온도 사라집니다

오늘 이 질문에 답합니다

적어둘 곳이 없어서 그렇습니다. 지금까지 값은 전부 메모리에만 있었습니
다

변수는 어디에 있었나

지금까지 만든 값이 살던 곳

01 `let todos = []` 라고 쓰면 메모리에 자리가 생깁니다

02 메모리는 브라우저 탭이 쓰는 임시 공간입니다

03 새로고침하면 탭이 처음부터 다시 시작합니다

04 그래서 메모리도 비워지고 변수도 사라집니다

탭을 연다

메모리에 자리를 받음



코드가 돈다

변수에 값이 쌓임



새로고침

자리를 반납하고 처음부터

localStorage, 브라우저가 주는 창고

탭을 닫아도 남는 자리

- 01 브라우저가 사이트마다 하나씩 주는 저장 공간입니다
- 02 새로고침해도, 브라우저를 껐다 켜도 남습니다
- 03 따로 설치할 것도 없고 그냥 씁니다
- 04 다만 이 브라우저 안에만 있습니다. 다른 기기에는 없습니다

localStorage.setItem("key", "value")

이름표를 붙여서 넣고, 이름표로 다시 꺼냅니다

콘솔에 넣고 새로고침해서 살아남는지 보기

```
localStorage.setItem("nickname", "홍길동");
```

```
// 여기서 새로고침 F5
```

```
localStorage.getItem("nickname");
```

할 일

콘솔에 한 줄씩. 새로고침한 다음 마지막 줄 실행

생각해볼 것

일반 변수에 넣었으면 어떻게 됐을까

남은 시간

08:00

쓰는 명령은 세 개뿐입니다

넣고, 꺼내고, 지우고

```
1 // 넣기
2 localStorage.setItem("nickname", "홍길동");
3
4 // 꺼내기
5 const name = localStorage.getItem("nickname");
6
7 // 지우기
8 localStorage.removeItem("nickname");
```

setItem

이름표와 값을 같이 넣습니다

getItem

이름표로 꺼냅니다

removeItem

이름표 하나만 지웁니다

파이썬에서도 하던 일입니다

이름만 다르고 하는 일은 같음

Python

```
with open("name.txt", "w") as f:
    f.write("홍길동")

with open("name.txt") as f:
    name = f.read()
```

JavaScript

```
localStorage.setItem("name", "홍길동");

const name = localStorage.getItem("name");
```

핵심 차이 파이썬은 파일에 적었고, 브라우저는 파일을 못 만들어서 localStorage 를 씁니다

값은 무조건 문자열이 됩니다

넣을 때 뭘 넣든 꺼내면 글자

01 숫자 3 을 넣으면 글자 "3" 이 나옵니다

02 true 를 넣으면 글자 "true" 가 나옵니다

03 배열을 넣으면 이상한 글자가 나옵니다

04 그래서 꺼낸 다음에 원래 모양으로 되돌려야 합니다

localStorage 는 글자만 담습니다

숫자도 배열도 객체도 전부 글자로 바뀌어 들어갑니다

넣은 것과 꺼낸 것이 다릅니다

타입이 바뀝니다

```
1 localStorage.setItem("count", 3);
2 const count = localStorage.getItem("count");
3
4 console.log(count);           // "3"
5 console.log(typeof count);    // "string"
6 console.log(count + 1);       // "31"
```

숫자를 넣었는데
꺼내면 글자입니다

count + 1
더하기가 아니라 이어붙이기가 됩니다

Number(count)
숫자로 되돌리려면 감싸야 합니다

숫자와 배열을 넣었다가 꺼내보기

```
localStorage.setItem("count", 10);  
localStorage.getItem("count") + 1;  
  
localStorage.setItem("list", [1, 2, 3]);  
localStorage.getItem("list");
```

할 일

한 줄씩 콘솔에서. 나온 값을 그대로 적어둘 것

생각해볼 것

배열은 왜 저렇게 나올까

남은 시간

08:00

JSON 으로 바꿔서 넣습니다

2일차에 배운 그것

- 01 JSON.stringify 는 배열이나 객체를 글자로 바꿉니다
- 02 JSON.parse 는 글자를 다시 배열이나 객체로 되돌립니다
- 03 넣을 때 stringify, 꺼낼 때 parse
- 04 2일차에 배운 것을 그대로 씁니다

배열
[1, 2, 3]



JSON.stringify
"[1,2,3]" 글자가 됨



JSON.parse
[1, 2, 3] 배열로 돌아옴

배열을 저장하고 꺼내는 전체

오늘 오후에 그대로 씁니다

```
1  const todos = [{ text: "장보기", done: false }];
2
3  // 넣을 때는 글자로 바꿔서
4  localStorage.setItem("todos", JSON.stringify(todos));
5
6  // 꺼낼 때는 다시 배열로
7  const saved = JSON.parse(localStorage.getItem("todos"));
8  console.log(saved[0].text);    // "장보기"
```

stringify 로 넣고

parse 로 꺼냅니다. 항상 같이 씁니다

한쪽만 하면

글자가 그대로 나오거나 에러가 납니다

이 여섯 줄이

오후 미션의 저장 기능 전부입니다

처음 열었을 때가 문제입니다

저장한 적이 없으면 뭐가 나올까

- 01 없는 이름표로 꺼내면 null 이 나옵니다
- 02 JSON.parse(null) 은 에러가 아니라 null 을 돌려줍니다
- 03 null 에는 push 도 map 도 못 씁니다
- 04 그래서 없으면 빈 배열로 시작하게 막아둡니다

```
JSON.parse(localStorage.getItem("todos")) || []
```

왼쪽이 null 이면 오른쪽 빈 배열을 씁니다

배열을 저장하고 새로고침해도 남게 만들기

```
const todos = ["장보기", "운동"];

// TODO 1 todos 를 localStorage 에 저장

// 새로고침

// TODO 2 꺼내서 배열로 되돌리고 콘솔에 찍기
// TODO 3 저장한 적 없는 이름표로도 꺼내보기
```

할 일

practice.html 을 열고 콘솔에서 직접

생각해볼 것

TODO 3 에서는 무엇이 나왔나

남은 시간

10:00

practice.html

여기까지를 한 번에 이어서 쳐봅니다

저장된 걸 눈으로 볼 수 있습니다

개발자 도구 Application 탭

01 F12 를 누르고 Application 탭으로 갑니다

02 왼쪽에서 Local Storage 를 펼칩니다

03 이름표와 값이 표로 보입니다

04 여기서 직접 고치거나 지울 수도 있습니다

F12 → Application → Local Storage

코드로 저장한 게 실제로 들어갔는지 확인하는 가장 빠른 방법

02 상태 하나, 화면 하나

사실 계속 이 구조로 쓰셨습니다

2일차부터 미션 세 개가 전부 같았음

01 2일차 대시보드. 데이터 배열 하나에 render 함수 하나

02 3일차 룰렛. 기록 배열 하나에 render 함수 하나

03 4일차 날씨. 받아온 데이터 하나에 render 함수 하나

04 이름만 달랐지 모양이 같았습니다

상태

지금 화면에 그려야 할 데이터 하나



render

상태를 보고 화면을 통째로 다시 그림



이벤트

상태를 바꾸고 render 를 다시 부름

코드로 보면 이렇게 생겼습니다

상태, 그리기, 바꾸기

```
1 // 1. 상태. 지금 화면에 있어야 할 것
2 let todos = [];
3
4 // 2. 그리기. 상태만 보고 화면을 만든다
5 function render() {
6   list.innerHTML = todos.map(todo => `<li>${todo.text}</li>`).join("");
7 }
8
9 // 3. 바꾸기. 상태를 고치고 다시 그린다
10 todos.push({ text: "장보기" });
11 render();
```

상태는 하나

화면의 진짜 내용은 여기에만 있습니다

render 는 상태만 봄

다른 데서 값을 안 가져옵니다

바꾸면 다시 그림

직접 화면을 고치지 않습니다

왜 통째로 다시 그리나

화면을 직접 고치면 생기는 일

화면을 직접 고치는 방식

```
// 추가할 때
list.innerHTML += "<li>장보기</li>";

// 지울 때는
// 몇 번째인지 찾아서
// 그 요소만 지워야 함
```

상태를 고치고 다시 그리는 방식

```
// 추가할 때
todos.push({ text: "장보기" });
render();

// 지울 때
todos.splice(index, 1);
render();
```

핵심 차이 왼쪽은 경우마다 코드가 늘어나고, 오른쪽은 상태만 고치면 화면은 알아서 맞습니다

상태를 바꾸고 다시 그려보기

```
let todos = ["장보기"];

function render() {
  document.querySelector("#list").innerHTML =
    todos.map(text => `<li>${text}</li>`).join("");
}

// TODO todos 에 하나 더 넣고 화면에 나오게 하기
```

할 일

practice.html 에서. render 를 다시 부르는 것을 잊지 말 것

생각해볼 것

render 를 안 부르면 어떻게 되나

남은 시간

08:00

저장은 언제 하면 되나

상태가 바뀔 때마다

- 01 추가할 때, 지울 때, 체크할 때 전부 저장해야 합니다
- 02 매번 따로 쓰면 언젠가 하나를 빠뜨립니다
- 03 상태가 바뀌면 render 를 부르기로 이미 정했습니다
- 04 그러면 save 도 render 옆에 두면 됩니다

상태를 바꿨으면 save 하고 render 한다

이 순서를 규칙으로 정해두면 빠뜨릴 일이 없습니다

저장 함수는 세 줄입니다

부르는 자리가 더 중요합니다

```
1 function save() {  
2   localStorage.setItem("todos", JSON.stringify(todos));  
3 }  
4  
5 // 상태를 바꾼 곳마다 이 두 줄을 같이  
6 todos.push({ text: "장보기", done: false });  
7 save();  
8 render();
```

save 는 세 줄

상태를 통째로 글자로 바꿔 넣습니다

항상 같이 부름

save 하고 render 하는 순서로 고정

빠뜨리면

화면은 바뀌는데 새로고침하면 돌아갑니다

페이지가 열릴 때 읽어옵니다

저장만 해두면 반쪽입니다

- 01 저장은 해줬는데 페이지를 열면 여전히 빈 화면입니다
- 02 시작할 때 localStorage 에서 읽어와야 합니다
- 03 읽어온 것을 상태에 넣고 render 를 한 번 부릅니다
- 04 저장한 적이 없으면 빈 배열로 시작합니다

```
function load() {  
  todos = JSON.parse(localStorage.getItem("todos")) || [];  
}  
  
load();  
render();
```

새로고침해도 남는 목록 만들기

```
// TODO 1  save 함수 만들기

// TODO 2  load 함수 만들기

// TODO 3  추가할 때 save 부르기

// TODO 4  페이지 맨 아래에서 load 하고 render 하기

// 다 하고 새로고침해서 남아 있으면 성공
```

할 일

practice.html 에서. 순서대로 하나씩

생각해볼 것

TODO 4 를 빠뜨리면 어떻게 되나

남은 시간

10:00

함수가 길어지면 못 고칩니다

AI 가 준 코드가 읽기 힘든 이유

01 한 함수가 서른 줄이 넘으면 어디를 고칠지 모릅니다

02 고치다가 다른 곳이 깨집니다

03 하는 일이 두 개 이상이면 나눌 때가 된 겁니다

04 이름만 봐도 뭘 하는지 알게 쪼갭니다

긴 함수 하나

받아오고, 계산하고, 화면 그리고, 저장까지



쪼갠다

getData, calculate, render, save



고칠 때

이름 보고 그 함수만 열면 됨

이름이 설명이 되게

주석을 안 써도 읽히는 코드

```
1 // 쪼개기 전
2 function handleAdd() {
3   const text = input.value.trim();
4   if (!text) return;
5   todos.push({ text: text, done: false });
6   localStorage.setItem("todos", JSON.stringify(todos));
7   list.innerHTML = todos.map(t => `<li>${t.text}</li>`).join("");
8   input.value = "";
9 }
```

하는 일이 네 개

읽기, 추가, 저장, 그리기

고칠 때

저장만 바꾸고 싶어도 전부 읽어야 함

쪼개면

save 와 render 를 따로 부르면 됩니다

쫄개면 이렇게 됩니다

줄 수는 늘었지만 고칠 곳을 바로 찾습니다

```
1 function handleAdd() {  
2   const text = input.value.trim();  
3   if (!text) return;  
4   addTodo(text);  
5   input.value = "";  
6 }  
7  
8 function addTodo(text) {  
9   todos.push({ text: text, done: false });  
10  save();  
11  render();  
12 }
```

handleAdd 는

입력칸을 다루는 일만 합니다

addTodo 는

목록을 바꾸는 일만 합니다

저장을 고치려면

save 함수 하나만 열면 됩니다

긴 함수를 세 조각으로 쪼개기

```
function handleAdd() {  
  const text = input.value.trim();  
  if (!text) return;  
  todos.push({ text: text, done: false });  
  localStorage.setItem("todos", JSON.stringify(todos));  
  list.innerHTML = todos.map(t => `<li>${t.text}</li>`).join("");  
  input.value = "";  
}
```

할 일

save 와 render 를 밖으로 빼고 handleAdd 를 짧게

생각해볼 것

함수 이름만 보고 무슨 일을 하는지 알 수 있나

남은 시간

08:00

03 콘솔 말고 중단점

console.log 만으로는 느립니다

지금까지 쓰던 방법의 한계

01 값이 이상하면 찍어보고, 또 찍어보고를 반복합니다

02 찍을 때마다 저장하고 새로고침해야 합니다

03 다 찾고 나면 지우는 것도 일입니다

04 코드를 잠깐 멈춰 세우면 한 번에 다 볼 수 있습니다

코드를 멈춰 세우고 그 자리에서 전부 확인

찍고 새로고침하기를 반복할 필요가 없습니다

중단점, 여기서 멈춰라

브레이크포인트

- 01 코드 줄 번호를 누르면 파란 표시가 붙습니다
- 02 그 줄에 도착하면 브라우저가 실행을 멈춥니다
- 03 멈춘 상태에서 모든 변수 값을 볼 수 있습니다
- 04 한 줄씩 넘기면서 값이 변하는 것도 볼 수 있습니다

줄 번호 클릭

파란 표시가 붙음



코드가 그 줄에 도착

실행이 멈춤



변수를 확인

그 시점의 모든 값이 보임

어디서 찍나

개발자 도구 Sources 탭

01 F12 를 누르고 Sources 탭으로 갑니다

02 왼쪽에서 내 HTML 파일을 찾아 누릅니다

03 코드가 보이면 줄 번호를 클릭합니다

04 새로고침하거나 버튼을 누르면 그 줄에서 멈춥니다

F12 → Sources → 파일 선택 → 줄 번호 클릭

네 단계면 끝납니다

찍는 자리가 중요합니다

의심되는 줄 바로 앞에

```
1 function handleAdd() {  
2   const text = input.value.trim();  
3   if (!text) return;  
4  
5   todos.push({ text: text, done: false });  
6   save();  
7   render();  
8 }
```

2번 줄에 찍으면

text 에 뭐가 들어왔는지 확인

5번 줄에 찍으면

push 직전 todos 상태 확인

고민되면

값이 이상해지기 직전 줄에 찍습니다

멈추면 이 세 군데를 봅니다

오른쪽 패널

- 01 Scope 는 지금 살아 있는 변수와 값 전부입니다
- 02 코드 위에 마우스를 올려도 값이 뜹니다
- 03 Console 탭에서 그 자리의 변수를 직접 쳐볼 수도 있습니다
- 04 Call Stack 은 지금 함수가 어디서 불렸는지입니다

Scope

지금 이 자리에서 살아 있는 변수 전부

마우스 올리기

코드 위에 올리면 그 값이 바로 뜸

Console

멈춘 자리에서 변수를 직접 쳐볼 수 있음

한 줄씩 넘겨봅니다

버튼은 세 개만 쓰면 됩니다

- 01** Resume 은 다음 중단점까지 그냥 갑니다
- 02** Step over 는 이 줄만 실행하고 다음 줄로 갑니다
- 03** Step into 는 함수 안으로 따라 들어갑니다
- 04** 값이 언제 이상해졌는지 찾을 때 Step over 를 씁니다

Resume, F8

다음 중단점까지 그냥 진행

Step over, F10

이 줄만 실행하고 다음 줄로

Step into, F11

함수 안으로 따라 들어가기

중단점을 찍고 변수 값 확인하기

```
function handleAdd() {  
  const text = input.value.trim();  
  todos.push({ text: text, done: false });  
  save();  
  render();  
}
```

할 일

2번 줄에 중단점을 찍고 추가 버튼을 눌러볼 것

생각해볼 것

멈춘 자리에서 todos 안에는 뭐가 들어 있나

남은 시간

10:00

반복문 안에서는 조건을 겁니다

매번 멈추면 못 씁니다

- 01 항목이 백 개면 백 번 멈춥니다
- 02 줄 번호를 오른쪽 클릭하고 Add conditional breakpoint 를 고릅니다
- 03 조건을 쓰면 그 조건일 때만 멈춥니다
- 04 `index === 50` 처럼 쓰면 쉼 번째에서만 멈춥니다

줄 번호 오른쪽 클릭 → Add conditional breakpoint

조건이 참일 때만 멈춥니다

오늘 쓸 것은 세 개뿐입니다

나머지는 나중에

01 중단점 찍기. 줄 번호 클릭

02 값 보기. Scope 패널이나 마우스 올리기

03 한 줄 넘기기. F10

04 이 세 개로 오늘 버그 세 개를 다 잡습니다

찍고, 보고, 넘긴다

다음 시간에 이걸로 AI 코드를 고칩니다

04 AI 코드 고치기

AI 는 돌아갈 것처럼 생긴 코드를 줍니다

1일차 오프닝에서 겪으셨습니다

- 01 문법은 맞고 구조도 그럴듯한데 안 돌아갑니다
- 02 AI 에게 다시 물으면 다른 곳을 고쳐서 줍니다
- 03 고쳐달라고 계속 물으면 코드가 점점 커집니다
- 04 어디가 문제인지 짚을 수 있으면 한 줄로 끝납니다

AI 를 쓰지 말라는 게 아닙니다

어디가 잘못됐는지 짚을 수 있어야 AI 도 제대로 씁니다

버그가 심어진 코드 세 개

전부 AI 에게 받은 것처럼 생겼습니다

- 01 1번은 값이 안 나오고 예러 문구만 뜹니다
- 02 2번은 버튼을 눌러도 아무 일이 없습니다
- 03 3번은 하나를 고쳤는데 다른 것까지 바뀝니다
- 04 원인은 각각 한 군데씩입니다

bug-1

온도가 안 나오고 예러 문구만

bug-2

버튼을 눌러도 아무 반응이 없음

bug-3

하나를 바꿨는데 다른 것도 같이 바뀜

이 순서로 하면 빠릅니다

코드부터 읽지 마세요

- 01** 증상을 한 문장으로 적습니다. 무엇이 안 되는가
- 02** 콘솔을 봅니다. 빨간 글씨가 있으면 줄 번호가 나옵니다
- 03** 그 근처에 중단점을 찍고 값을 확인합니다
- 04** 값이 이상해진 첫 지점이 원인입니다

증상 적기

무엇이 안 되는가를 한 문장으로



콘솔 보기

빨간 글씨와 줄 번호



중단점 찍기

값이 이상해진 첫 자리를 찾는다

bug-1 을 고치고 메모 두 줄 채우기

```
// 증상
//   온도가 안 나오고 에러 문구만 뜸

// 할 일
//   1. 원인이 되는 곳 한 군데를 찾아서 고치기
//   2. 아래 메모 두 줄 채우기

// 고친 곳 (몇 번째 줄, 어떤 함수):
// 왜 그게 원인이었나 (한 줄):
```

할 일

debug/bug-1-weather-card.html 을 열고. AI 사용 허용

생각해볼 것

AI 가 고쳐줬다면 왜 그게 원인인지 설명할 수 있나

남은 시간

15:00

오늘 만날 에러

오후에 반드시 한 번은 봅니다

에러 메시지	원인	해결
Unexpected token o in JSON	JSON.parse 에 객체를 넣음	stringify 한 글자를 넣어야 함
Cannot read properties of null	저장한 적 없는 키를 꺼냄	[] 로 기본값을 줄 것
todos.map is not a function	배열이 아니라 글자가 들어옴	JSON.parse 를 빠뜨렸는지 확인

오전에 배운 것

오후에 그대로 씁니다

01 localStorage 는 stringify 로 넣고 parse 로 꺼낸다

02 상태를 바꾸면 save 하고 render 한다

03 안 되면 중단점을 찍고 값을 본다

04 이 세 줄이 오후 미션의 전부입니다

save 하고 render 한다

오늘 오후 내내 이 한 줄을 씁니다

크롬 새 탭을 내가 만든 걸로

새 탭을 열 때마다 내 할 일 목록이 뜹니다

기본, 전원 완료 목표

- 할 일을 추가하고 화면에 나오게
- 체크하면 완료 표시가 되게
- 새로고침해도 남아 있게

심화, 기본을 마친 뒤

- 삭제 버튼 달기
- 완료한 것만 숨기기

확장 프로그램 만드는 법

세 단계면 끝납니다

- 01 폴더를 하나 만들고 파일 세 개를 넣습니다
- 02 크롬 주소창에 chrome://extensions 를 칩니다
- 03 오른쪽 위 개발자 모드를 켭니다
- 04 압축해제된 확장 프로그램을 로드해서 폴더를 고릅니다

manifest.json

확장 프로그램 설명서. 6줄짜리

newtab.html

새 탭에 보일 화면

newtab.js

동작하는 코드

설명서 파일은 여섯 줄입니다

manifest.json

```
1 {  
2   "manifest_version": 3,  
3   "name": "나의 새 탭",  
4   "version": "1.0",  
5   "chrome_url_overrides": { "newtab": "newtab.html" }  
6 }
```

manifest_version

3 으로 고정합니다

chrome_url_overrides

새 탭을 내 파일로 바꿉니다

파일 이름

manifest.json 이어야 합니다. 오타 주의

지금까지와 딱 하나가 다릅니다

확장 프로그램에서는 script 안에 코드를 못 씁니다

```
1 <!-- 지금까지 이렇게 썼습니다. 확장에서는 안 됩니다 -->
2 <script>
3   const todos = [];
4 </script>
5
6 <!-- 확장에서는 파일을 따로 만들어 불러와야 합니다 -->
7 <script src="newtab.js"></script>
```

증상

화면은 뜨는데 아무것도 동작하지 않습니다

콘솔에

Content Security Policy 라는 빨간 글씨가 뜹니다

해결

코드를 newtab.js 로 옮기고 src 로 불러옵니다

이제 내 할 일 목록은 새로고침해도 남습니다. 그런데 집에 있는 노트북에서 열면 왜 비어 있을까요

오늘 배운 것

세 줄로 정리

01 localStorage 로 저장하고 불러오기

02 상태를 바꾸면 save 하고 render 하기

03 중단점을 찍어 값이 이상해진 자리 찾기

TOMORROW

오전, 지금까지 배운 것을 하나로 오후, 각자 만들 것 정하고 시작